

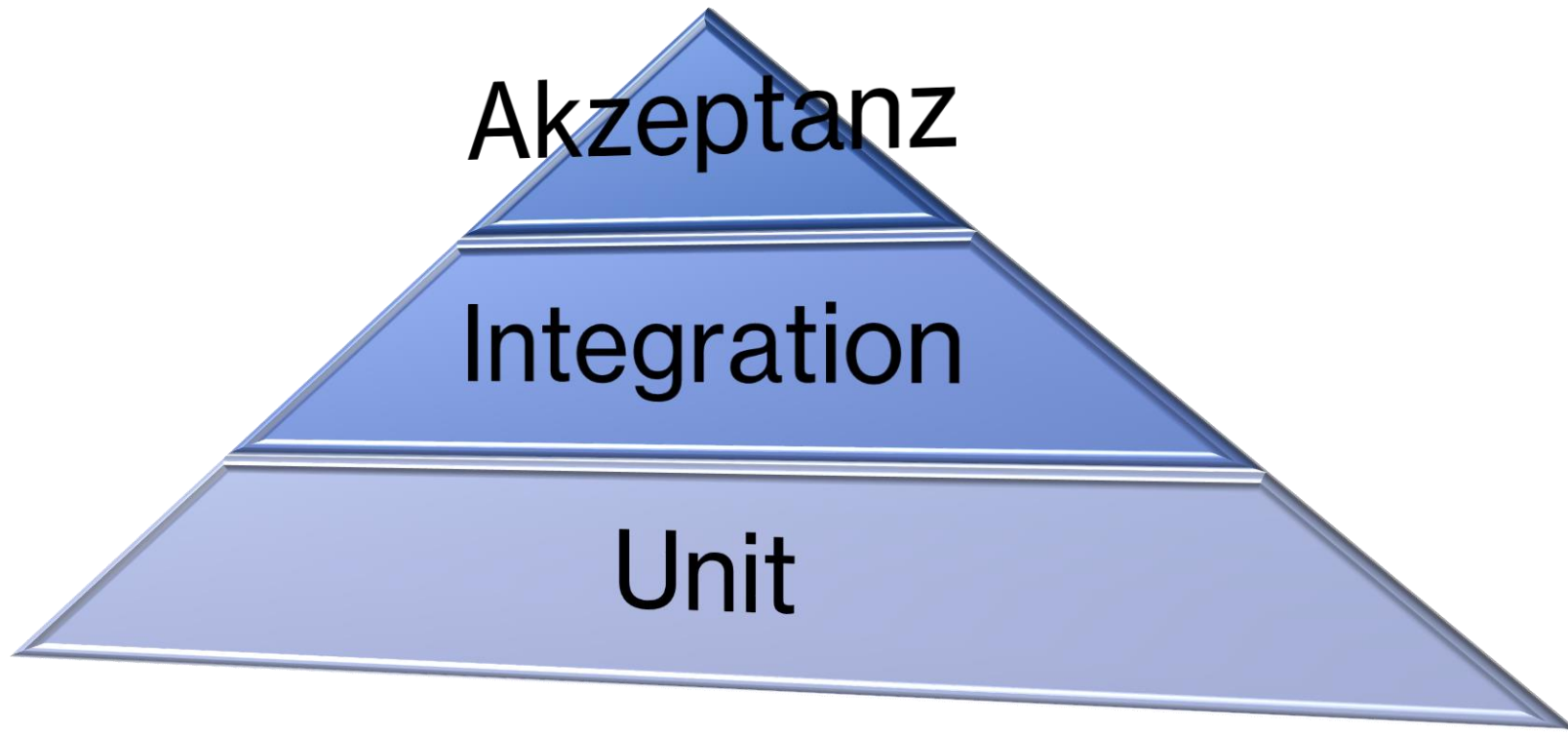


Testing

Jasmine Unittest

End-to-End

Testpyramide

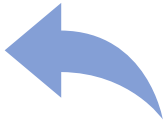


Default Angular Unittest Framework < V17

- Karma
 - TaskRunner (Start Testbed, Reporting Testresults)
- Jasmine
 - API/Framework for the individual Test (Expectations , ...)

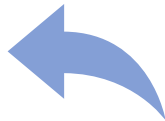
Konfiguration Karma package.json

```
"devDependencies": {  
  ..  
  "@types/jasmine": "~4.3.0",  
  "jasmine-core": "~4.6.0",  
  "karma": "~6.4.0",  
  "karma-chrome-launcher": "~3.2.0",  
  "karma-coverage": "~2.2.0",  
  "karma-jasmine": "~5.1.0",  
  "karma-jasmine-html-reporter": "~2.1.0",  
  ..}  
}
```



Konfiguration Karma angular.json

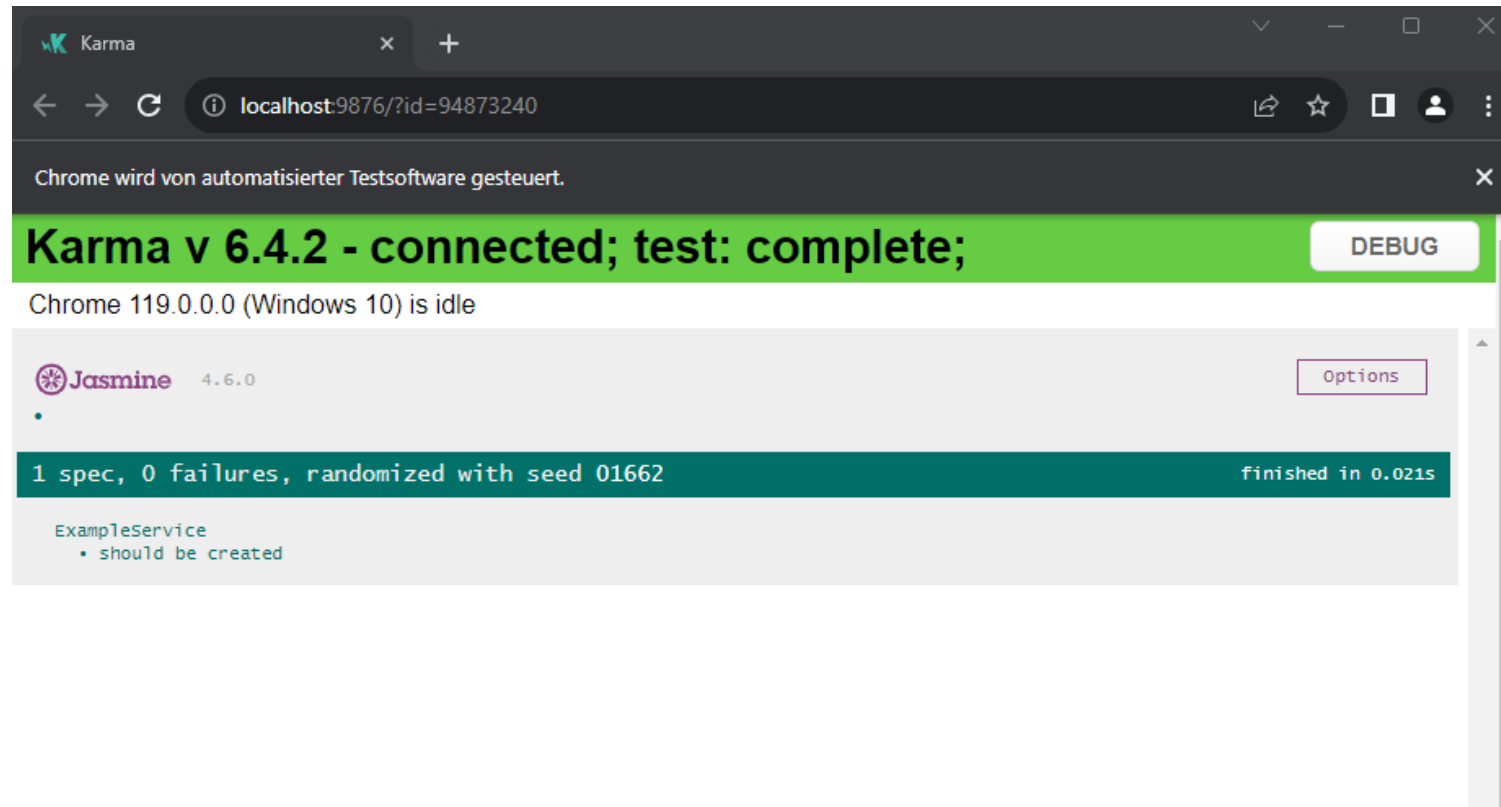
```
"test": {  
  "builder": "@angular-devkit/build-angular:karma",  
  "options": {  
    "polyfills": [  
      "zone.js",  
      "zone.js/testing"  
    ],  
    "tsConfig": "tsconfig.spec.json",  
    "assets": [  
      "src/favicon.ico",  
      "src/assets"  
    ],  
    "styles": [  
      "src/styles.css"  
    ],  
    "scripts": []  
  }  
}
```



Konfiguration Karma tsconfig.spec.json

```
/* To learn more about this file see: https://angular.io/config/tsconfig. */  
{  
  "extends": "./tsconfig.json",  
  "compilerOptions": {  
    "outDir": "./out-tsc/spec",  
    "types": [  
      "jasmine"  
    ]  
  },  
  "include": [  
    "src/**/*.spec.ts",  
    "src/**/*.d.ts"  
  ]  
}
```





Karma Runner

> ng test

Jasmine Vokabular

Suites – describe(String, function)

- Eine Suite definiert eine Menge von Spezifikationen

Specs – it(string, function)

- 1 Test bestehend aus Titel
- und einer Funktion mit dem Test -> Erwartungen(Expectations).

Expectations expect(actual).toBe(expected)

- Erwartung - technische Implementierung deines Tests → true oder false ausgewertet.

Matchers

- vordefinierte Helferfunktionen für gemeinsame Expectations. z.B.: toBe(expected), toEqual(expected).
- komplette Liste ([GitHub - JamieMason/Jasmine-Matchers: Write Beautiful Specs with Custom Matchers for Jest and Jasmine](#)).

Einfaches Beispiel – Testsubjekt

ExampleService

```
describe('ExampleService', () => {  
  let service: ExampleService;
```

Suite

```
    beforeEach(() => {  
      TestBed.configureTestingModule({});  
      service = TestBed.inject(ExampleService);  
    });
```

Setup

```
    it('should be created', () => {  
      expect(service).toBeTruthy();  
    });  
  });
```

Expectation

Matcher

Setup, Teardown

beforeEach
afterEach

- einmal pro Spec (it)

beforeAll
afterAll

- einmal pro Test-Suite (describe)

Setup initialisiere Testwelt

- beforeEach(() => {
 TestBed.configureTestingModule({}); Setup TestBed mittels Config
 Object
 service = **TestBed**.inject(ExampleService);
});

Expectation

-

Beschreibung

- `it('should be created', () => {
 expect(service).toBeTruthy();
});`

Testcode mit matcher

UnitTest Component

```
@Component({  
  selector: 'app-greeter',  
  templateUrl: './greeter.component.html',  
  styleUrls: ['./greeter.component.css']  
})  
export class GreeterComponent  
implements OnInit{  
  public name : string = "";
```

- <h1>Hello {{name}}</h1>

12/13/2023

UnitTests Component - Setup

```
let component: GreeterComponent;
let fixture: ComponentFixture<GreeterComponent>; Teststeuerobjekt für Component

beforeEach(() => {
  TestBed.configureTestingModule({
    declarations: [GreeterComponent] Config Object wie im Module
  });
  fixture = TestBed.createComponent(GreeterComponent);

  component = fixture.componentInstance;
  componentRootElement = fixture.nativeElement; HTML Element der Component
  debugElement = fixture.debugElement;

  fixture.detectChanges(); ChangeDetection Trigger
});
```

UnitTests Component – Set

```
it('should show Hello <name>', async() => {  
  component.name = 'Tester';           Set Property  
  
  fixture.detectChanges();             Trigger Change Detection  
  
  fixture.whenStable().then(() => {     Warte bis alle Async Calls beendet sind  
    expect(componentRootElement.querySelector('h1')?.innerText).toBe('Hello Tester');  
    expect(debugElement.query(By.css('h1')).nativeElement.innerText).toBe('Hello Tester');  
  
  });
```

Dependency Injection

```
export class GreeterComponent implements
OnInit{
  public name : string = "";

  constructor(private service: GreeterService)
{}

  public ngOnInit(): void {
    this.name = this.service.requestName();
  }
}
```

```
export class GreeterService {

  constructor() { }

  public requestName():string{
    return "some example name"
  }
}
```

12/13/2023

Mocking Dependencies

```
let greeterServiceSpy = jasmine.createSpyObj("GreeterService", ['requestName']);
```

```
beforeEach(() => {
```

```
  TestBed.configureTestingModule({  
    declarations: [GreeterComponent],  
    providers: [{provide: GreeterService, useValue: greeterServiceSpy}]  
  });
```

```
  greeterServiceSpy.requestName.and.returnValue("Tester")
```

```
  fixture = TestBed.createComponent(GreeterComponent);  
});
```

Async result

```
<p *ngFor="let name of names">
  {{name}}
</p>
```

```
export class GreeterComponent {
  public names : string[] = [];

  constructor(private service: GreeterService) {}

  public requestNames(): void {
    this.service.requestNames().subscribe(result =>
      this.names = result
    );
  }
}
```

```
export class GreeterService {

  public requestNames():Observable<string[]>{
    return //HTTP CALL...
  }
}
```

```
});
```

Aync result

```
greeterServiceSpy.requestNames.and.returnValue(of(["Tester1", "Tester2"]));
```

```
...
```

```
it('should show nothing at the beginning', async() => {
```

```
    fixture.detectChanges();
    fixture.whenStable().then(() => {
        expect(componentRootElement.querySelector('p')).toBeNull();
    });
```

```
it('after requesting data 2 p-elements should be visible', async() => {
```

```
    component.requestNames();
    fixture.detectChanges();
    fixture.whenStable().then(() => {
        expect(componentRootElement.querySelectorAll('p')).toHaveLength(2);
    });
```

Throw Error

```
greeterServiceSpy.requestNames.and.returnValue(throwError(() => new Error("TestError")));
```

Trigger Eventhandling

```
<p *ngFor="let name of names" (click)="setSelectedName(name)">
  {{name}}
</p>
```

```
@Output()
public selected: EventEmitter<string> = new EventEmitter<string>();
```

```
public setSelectedName(name: string): void {
  this.selected.emit(name)
}
```

```
let selectedName : string = "";
component.selected.pipe(first()).subscribe((name) => (selectedName = name));
```

```
var allPElements = fixture.debugElement.queryAll(By.css('p'));
allPElements.at(0)?.triggerEventHandler('click');
```

```
expect(selectedName).toEqual("Tester1");
```

Übung 1

- Teste die Klasse TrainingListComponent
 - Mocke den Service und gebe 0, 1, 2 Trainings zurück
 - Prüfe ob die Liste korrekt aufgebaut
- Teste die TakePlaceSoon Pipe
 - Prüfe mit verschiedenen Daten Vergangenheit, Zukunft (in „soon“a), Zukunft (nicht „soon“)

Cypress

End2End Tests

Installation Cypress

ng e2e

Cannot find "e2e" target for the specified project.

You can add a package that implements these capabilities.

For example:

Cypress: ng add @cypress/schematic

Nightwatch: ng add @nightwatch/schematics

WebdriverIO: ng add @wdio/schematics

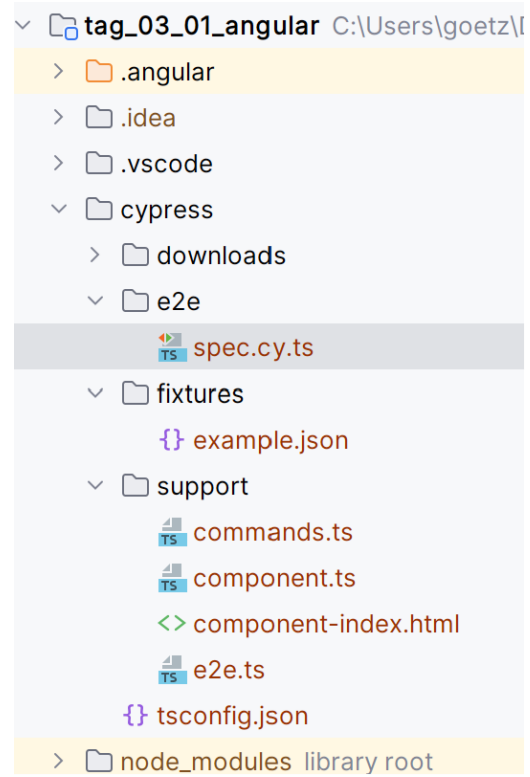
Puppeteer: ng add @puppeteer/ng-schematics

Would you like to add a package with "e2e" capabilities now? Cypress

i Using package manager: npm

...

Konfig



Bestandteile eines Tests

```
describe('My First Test', () => {
```

Suite

```
  it('Visits the initial project page', () => {
```

Specs

```
    cy.visit('/')
```

```
    cy.pause();
```

```
    cy.get("li").first().should("have.class","selected");
```

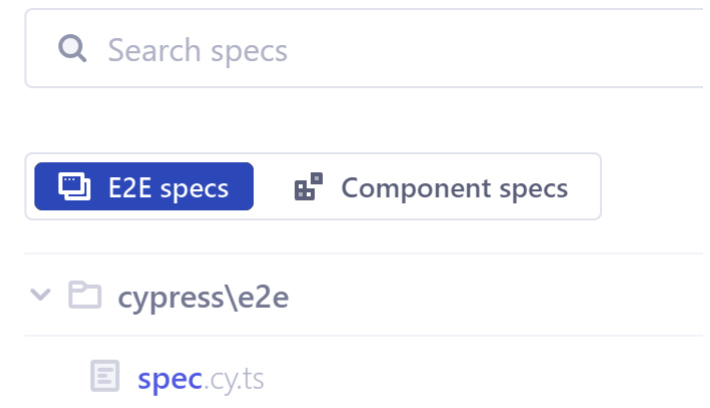
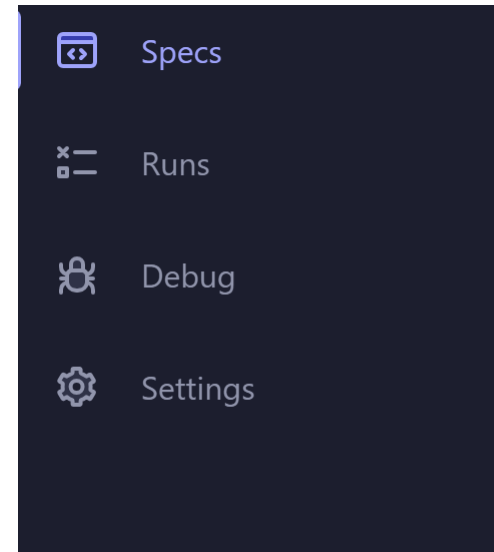
Matcher, Expectation

```
  })
```

```
}
```

Übung 2

- Installieren sie Cypress mittels ng e2e
- Passen sie die generierte Datei spec.cy.ts
- Ändern sie
 - `cy.contains('app ís running')` in
 - `cy.contains('Hallo Training Manager App')` ab
- Starten sie ng e2e und navigiere zu
 - spec.cy.ts



Cypress Elements – DSL

- Elemente definieren die structure und Verhalten einer Seite.
- Durch das selektieren und interagieren mit diesen, kann man funktionale Tests schreiben.
- Beispiel ID Selektor
 - `cy.get('#username').type('user@angularseminar.com');`
 - `cy.get('#password').type('password');`
- Beispiel Attribut Selektor
 - `cy.get('[formControlName=username]').type('user@angularseminar.com');`
- Beispiel Element Selektor
 - `cy.get('ul');`

Cypress Elements – Element API

Funktion	Bedeutung
.type()	Ausfüllen eines Textfeldes
.click()	Klicken / Selektieren
.first()	Falls element NodeListe, gib erstes
.focus()	Setzt den Fokus auf das Element
.should()	Matcher siehe nächste Folie

Assertions und Matchers

- Die Funktion `should()` an einem Element leitet den Prüfschritt ein
- `cy.get("appTrainingList").should("exist");`
- `cy.get(".selected").should("not.exist");`
- `cy.get(".selected").should("be.visible");`
- `cy.get("li").first().should("have.class","selected");`
- `cy.get("li").first().should("not.have.value","Angular");`
- `cy.url().should("contain","greeting");`

Übung 3

- Erstellen Sie eine neue Test Methode navigation_works
- Hier soll der erste Eintrag in der Trainingsliste geklickt werden
- Danach soll geprüft werden, dass
 - Die App auf der Detailseite ist
 - Mittels URL Matcher
 - Und prüfen ob nextRun Date angezeigt wird